

快速体验

KE1 终端开发环境主要准备工作包含以下 3 部分：

1. 拥有一定的C语言基础，否则寸步难行！（必选）
2. 安装 STM32 官方免费开发工具 STM32CubeIDE（必选）
 - 官方途径 [下载最新版本](#) (需要注册，下载速度快)
 - [网盘下载](#) 提取码: yam6，找到en.st-stm32cubeide_xx下载（版本可能不是最新，windows x64平台）
3. 安装版本控制软件 [Git](#) 和 [TortoiseGit](#) 可以更加方便的获取我们提供的最新源代码，熟练使用Git也是程序员必备技能之一（可选）
4. 下载(解压) [KE1综合实例源码](#) KE1South（必选）



5. 下载 [其他例程源码](#)（可选）

开发环境安装好后，您就可以体验KE1综合例程了。整个过程可以概括为如下 6 步：

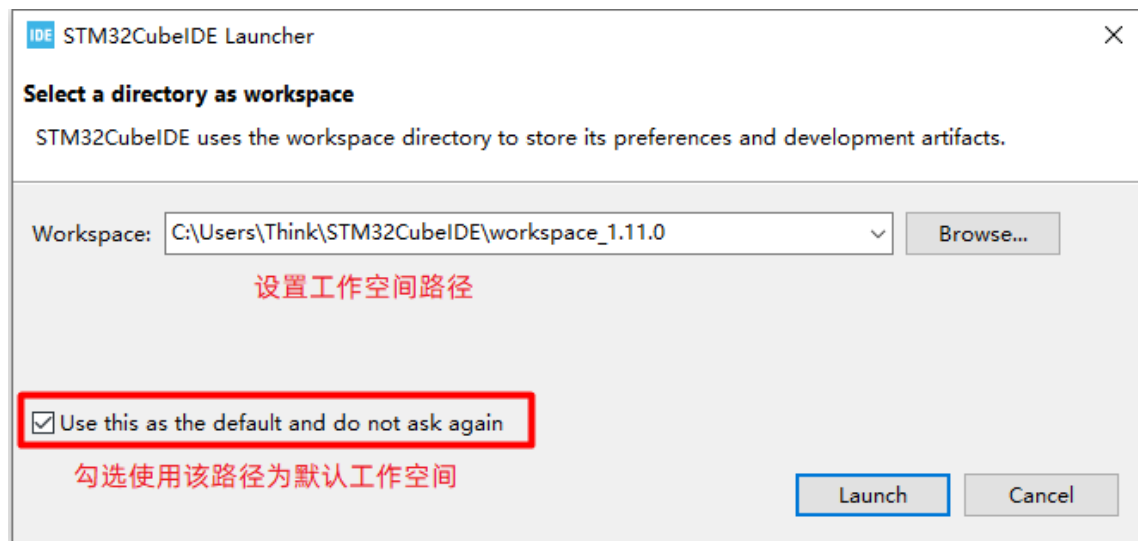
1. 启动开发工具 STM32CubeIDE
2. 导入 KE1综合例程源码（KE1South）
3. 编译源码
4. 通过 ST-Link 调试器将KE1与电脑进行正确连接
5. 启动 STM32CubeIDE 调试功能，运行程序或调试代码
6. 通过本平台提供的微信小程序或应用 APP 远程监控 KE1 终端

STM32CubeIDE 安装

STM32CubeIDE 的安装唯一需要注意的是，安装路径、设置工作空间(workspace)以及工程源码路径中**不能有中文或者其它特殊符号**，安装步骤默认点“下一步”即可。

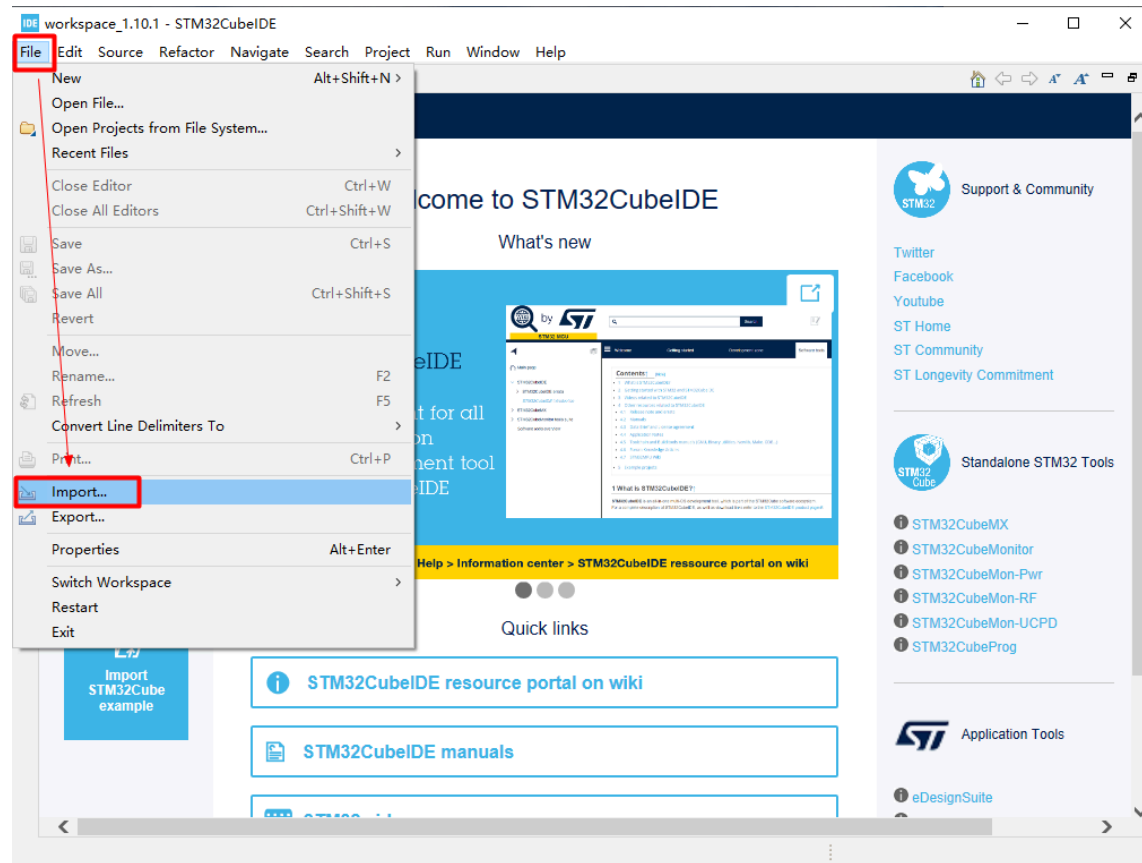
本例使用的 STM32CubeIDE 版本为"1.10.1"，安装路径如
"D:\ST\STM32CubeIDE_1.10.1\STM32CubeIDE" (仅供参考)

首次启动软件会提示设置工作空间，如
"C:\Users\Think\STM32CubeIDE\workspace_xx.xx.xx" (仅供参考)

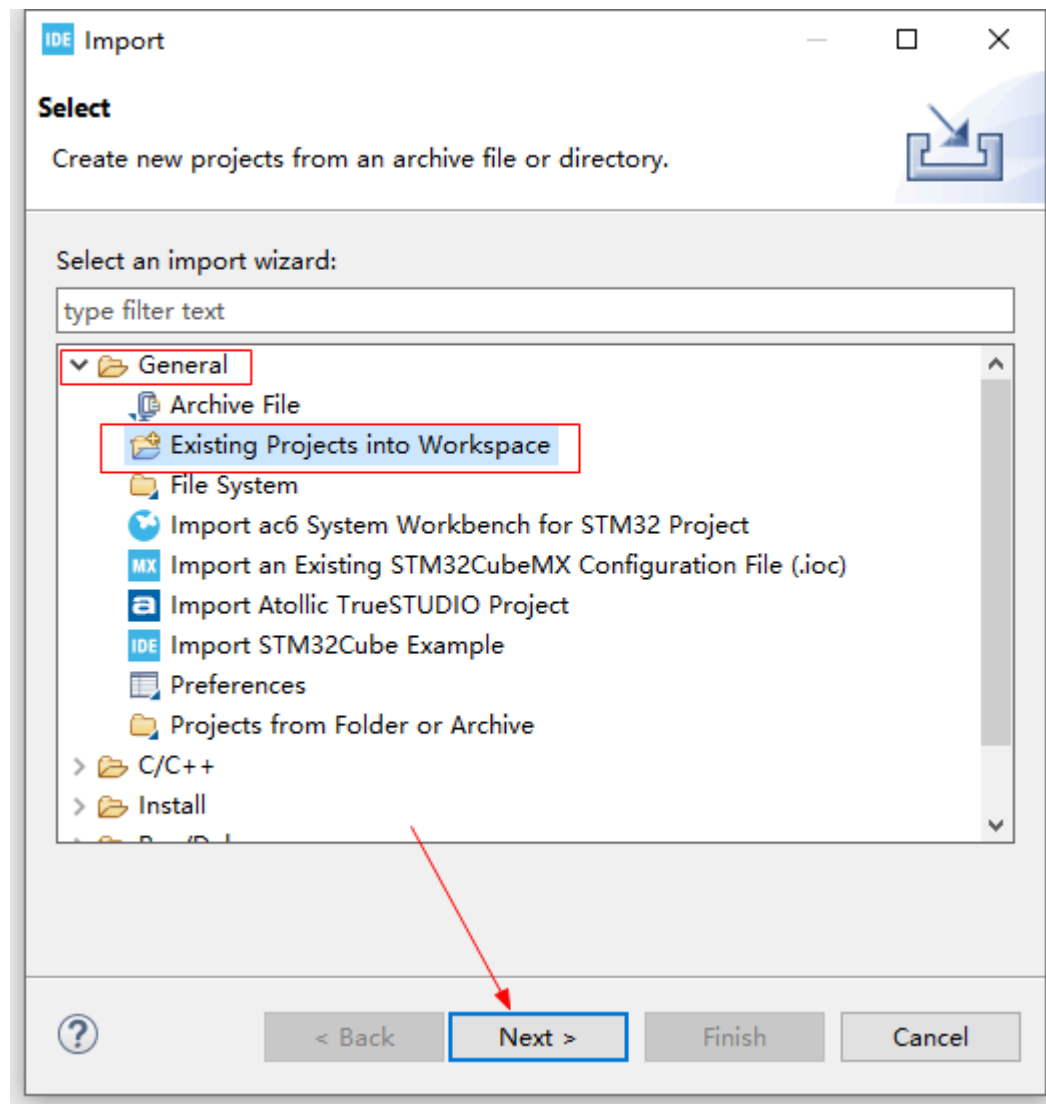


STM32CubeIDE 导入工程

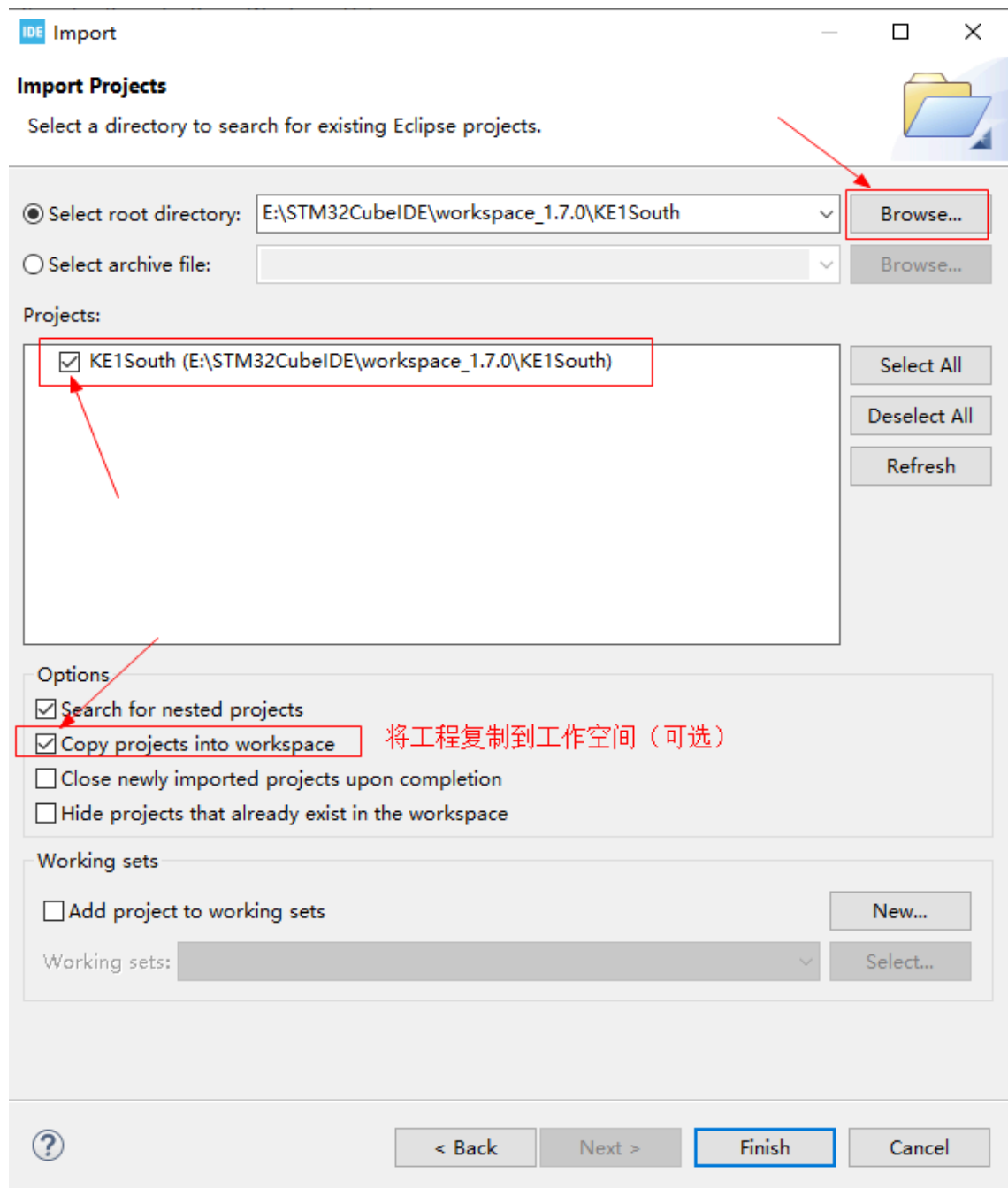
1. 导入KE1综合例程源码（例如本例KE1South）



2. 选择导入已存在的工程到工作空间

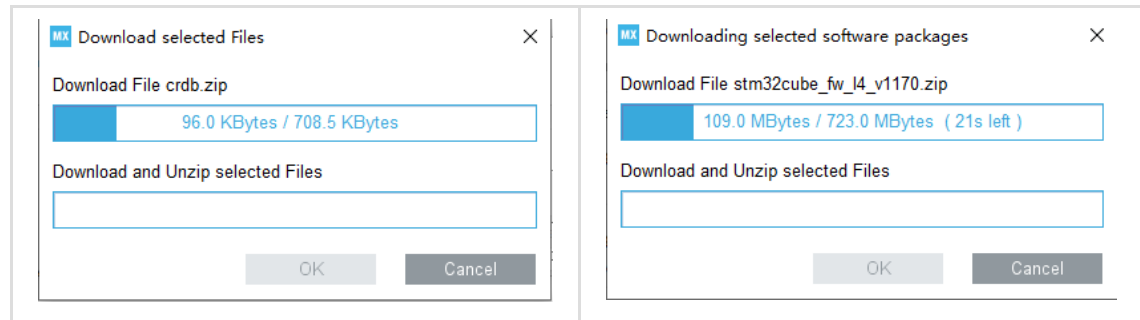


3. 点击(Browse)选择KE1South源码工程目录，勾选工程文件，点击“Finish”完成导入工作

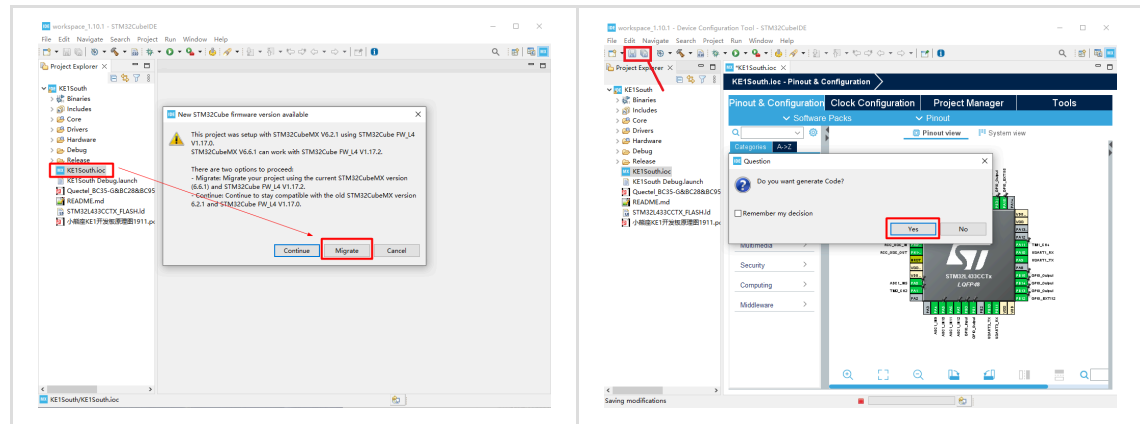


可将工程源码导入到默认工作空间(workspace)

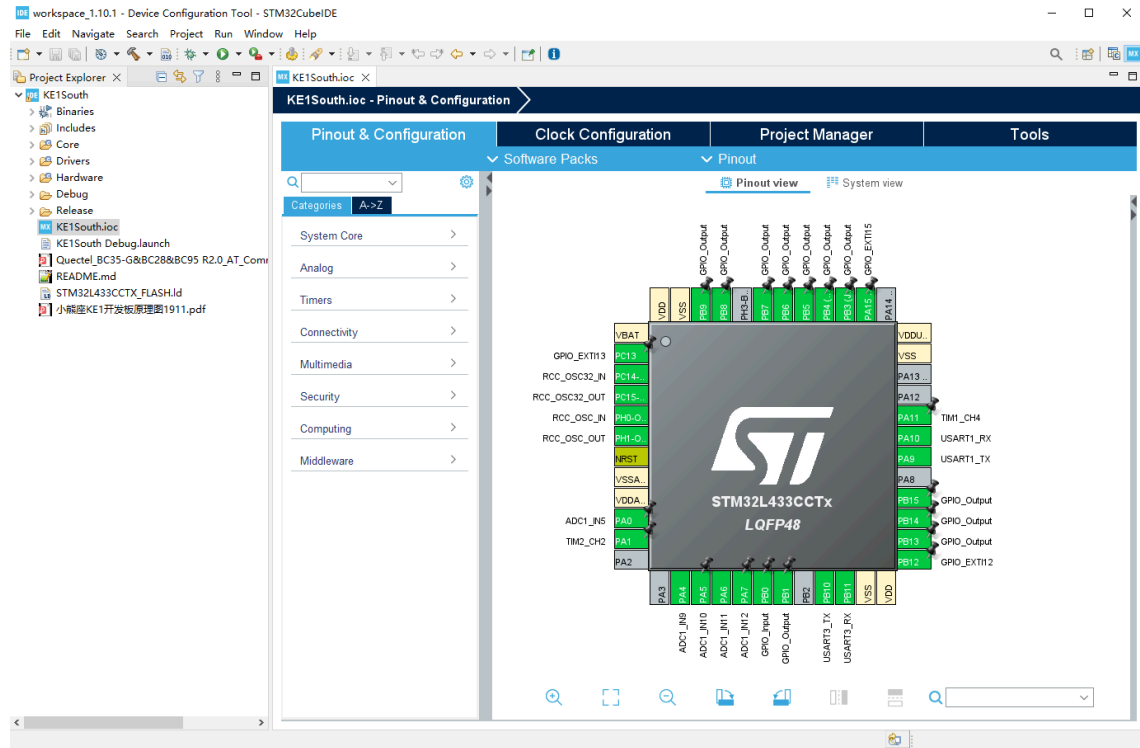
4. 首次启动可能会进行如下初始化提示，并自动下载对应芯片的固件开发包，等待完成即可。



5. 双击KE1South.ioc，打开工程配置界面，首次启动可能提示版本不匹配，选择“Migrate”进行迁移操作，并保存修改，此时会提示重新生成代码，点击"OK"，并等待自动完成即可。

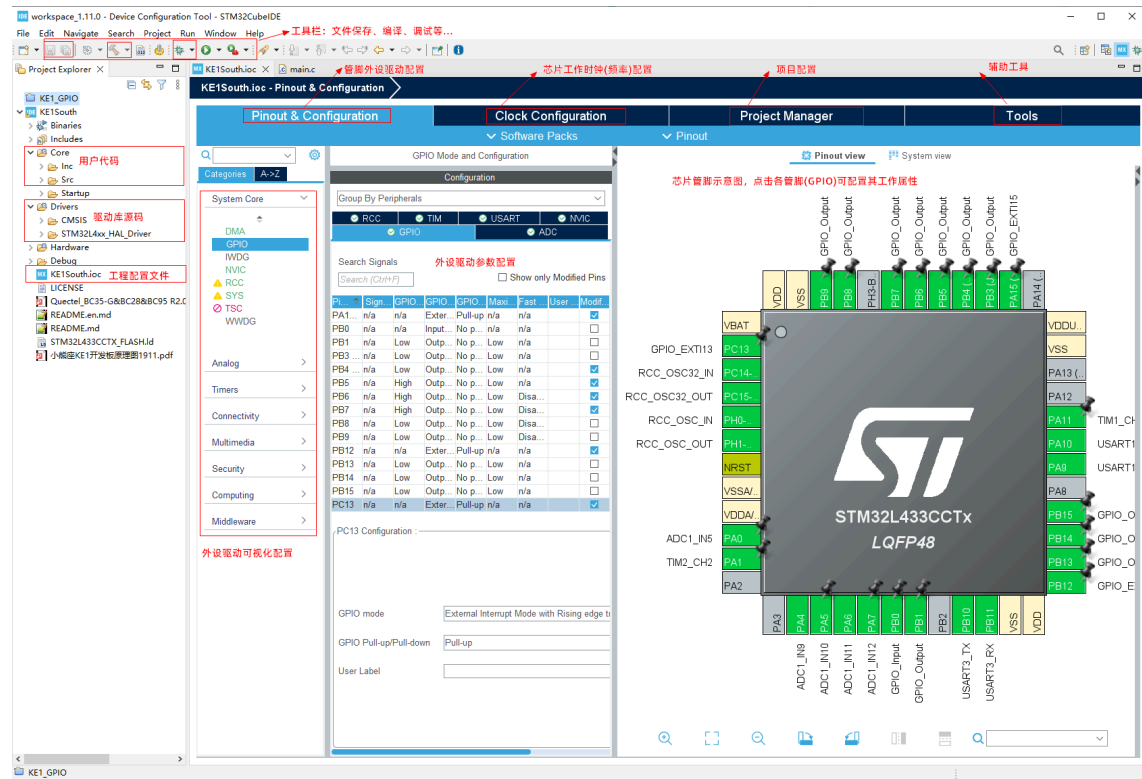


6. 导入完成



STM32CubeIDE 工具简介

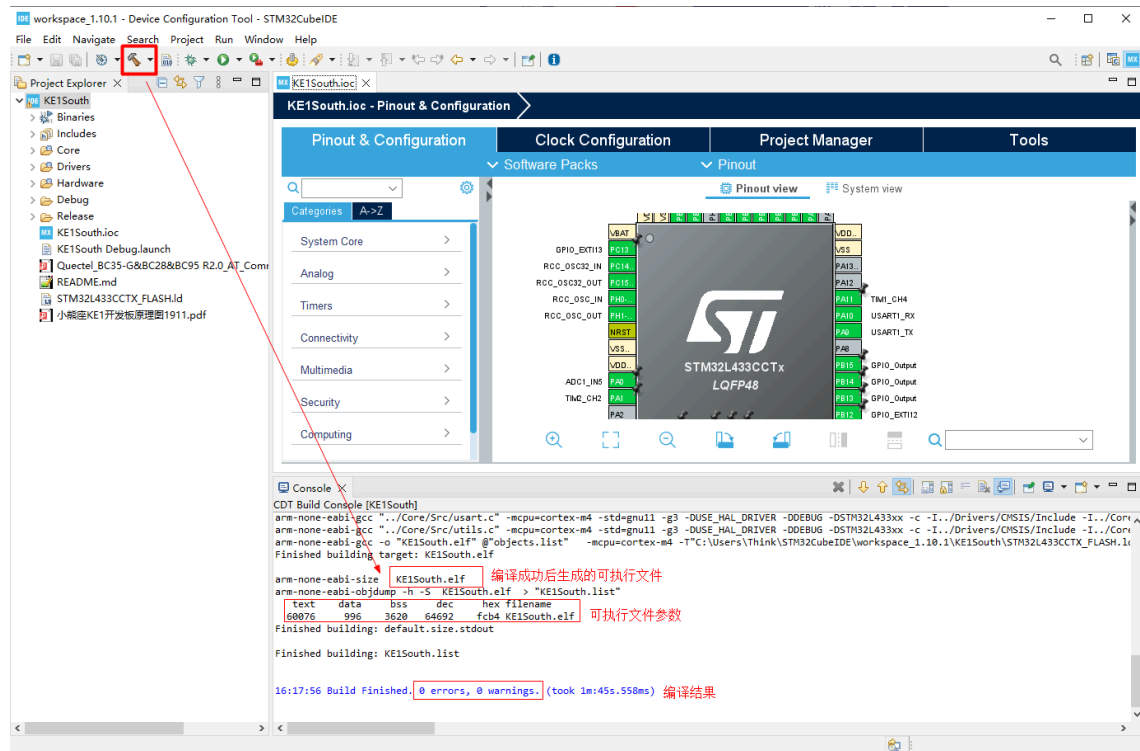
界面功能区简介



基本功能简介（编译、调试、运行）



点击“编译”后，在Console界面会输出编译过程和结果，提示"0 errors, 0 warnings."即表示编译成功，如果有告警(warning),可酌情处理。本案例编译出来的可执行文件为"KE1South.elf"



markup

目标可执行文件参数说明：

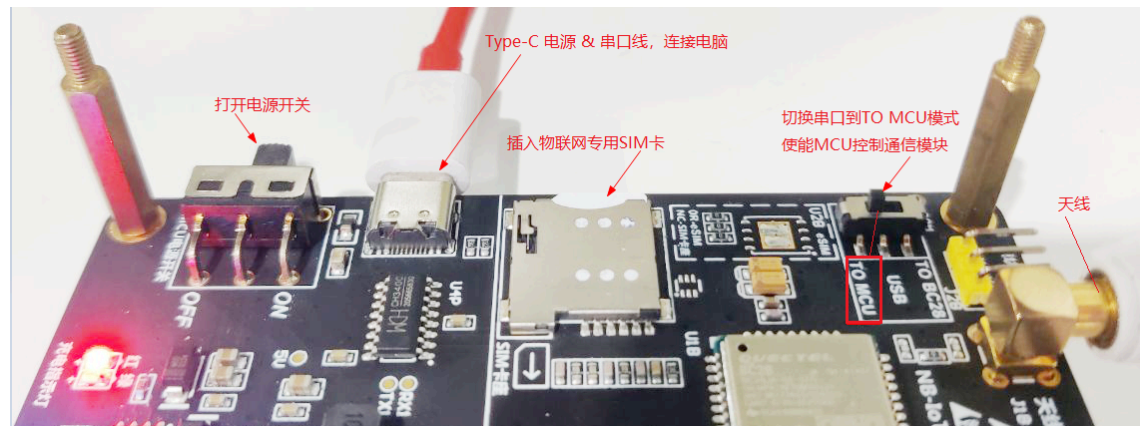
- .text: 代码段，代码和常量大小
- .data: 数据段，已初始化的全局/静态变量
- .bss: 未初始化的全局/静态变量
- .dec: 上述3者的总和
- .hex: dec的16进制显示

STM32CubeIDE 使用 ST-Link 调试代码

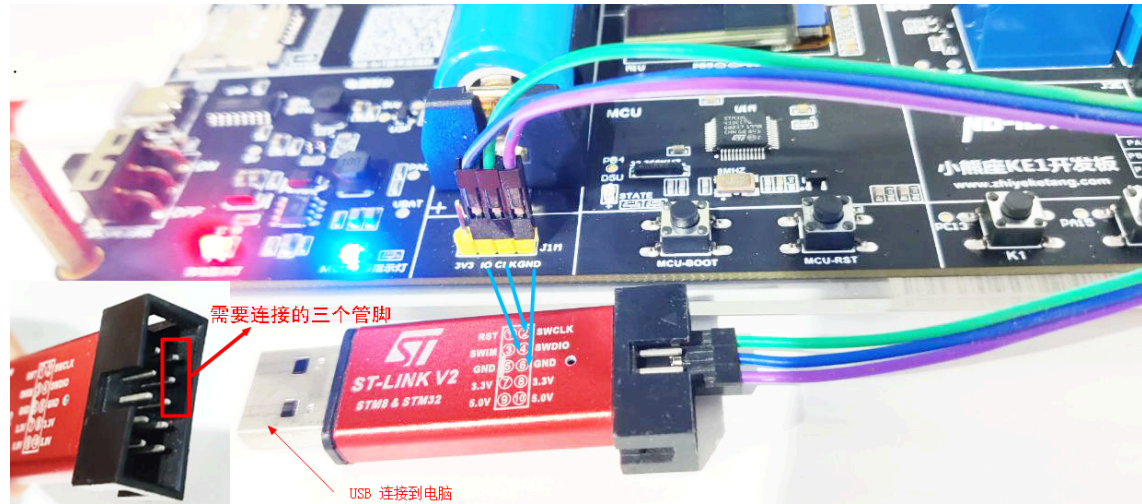
在进入 KE1 综合例程调试前，请完成以下步骤

- 插入物联网 SIM 卡，并安装天线
- 切换串口通道为 TO MCU 位置，使能 MCU 控制通信模组
- 将 KE1 Type-C电源&串口线与电脑连接
- 连接 ST-Link 调试器(参考第2步)
- 启动 KE1 电源

1. 按照下图提示完成 KE1 准备工作



2. 将 KE1 与 ST-Link 调试器按下图所示正确连接



markup

KE1 与 ST-Link 线序连接如下：

GND <----> GND

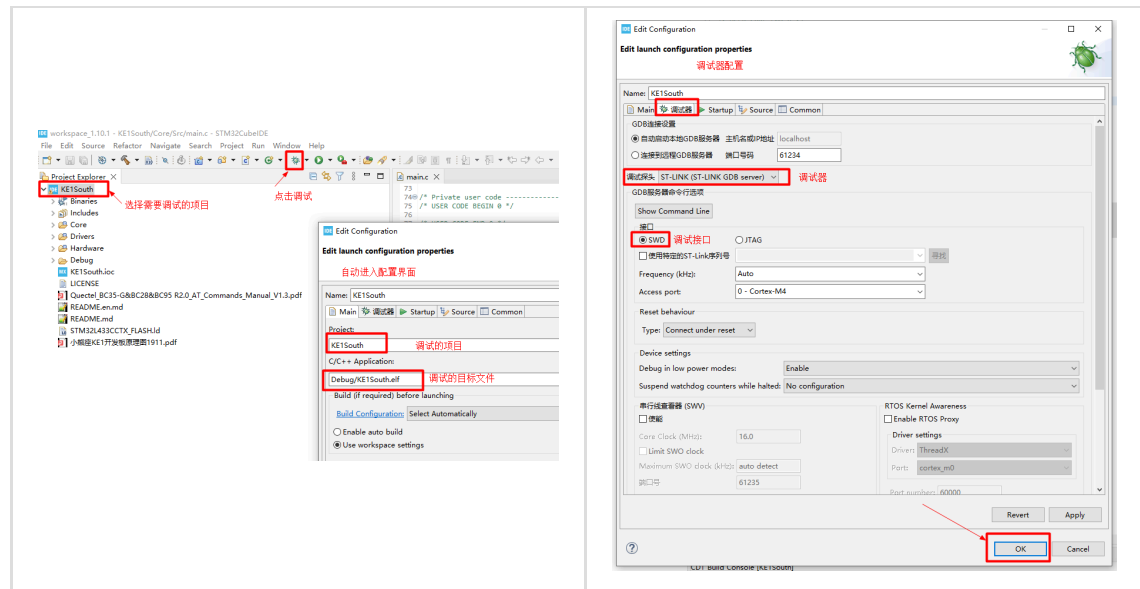
IO <----> SWDIO

CLK <----> SWCLK

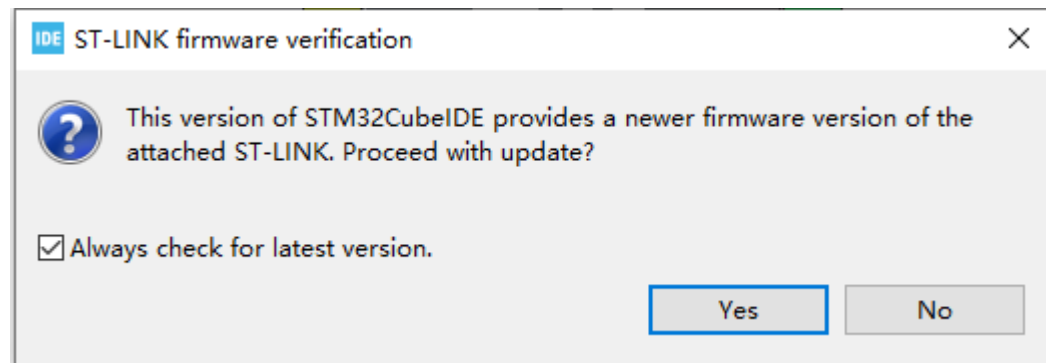
另外，ST-Link调试器的USB口需要插入您的工作电脑上

启动调试

点击调试按钮，首次将自动进入调试配置界面，按照如图配置后，点击“OK”，启动调试

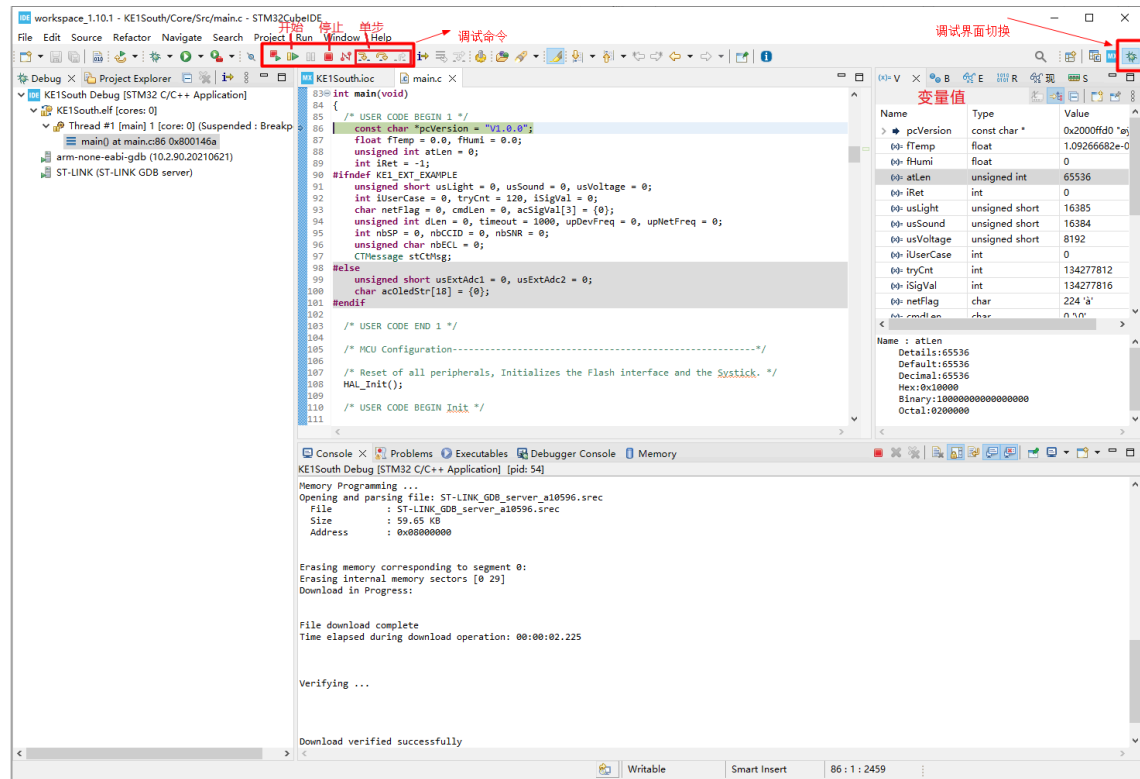


首次启动调试可能会提示有新的ST-Link固件需要升级，可参考【ST-Link调试帮助】章节进行升级。升级成功后，再次启动调试。



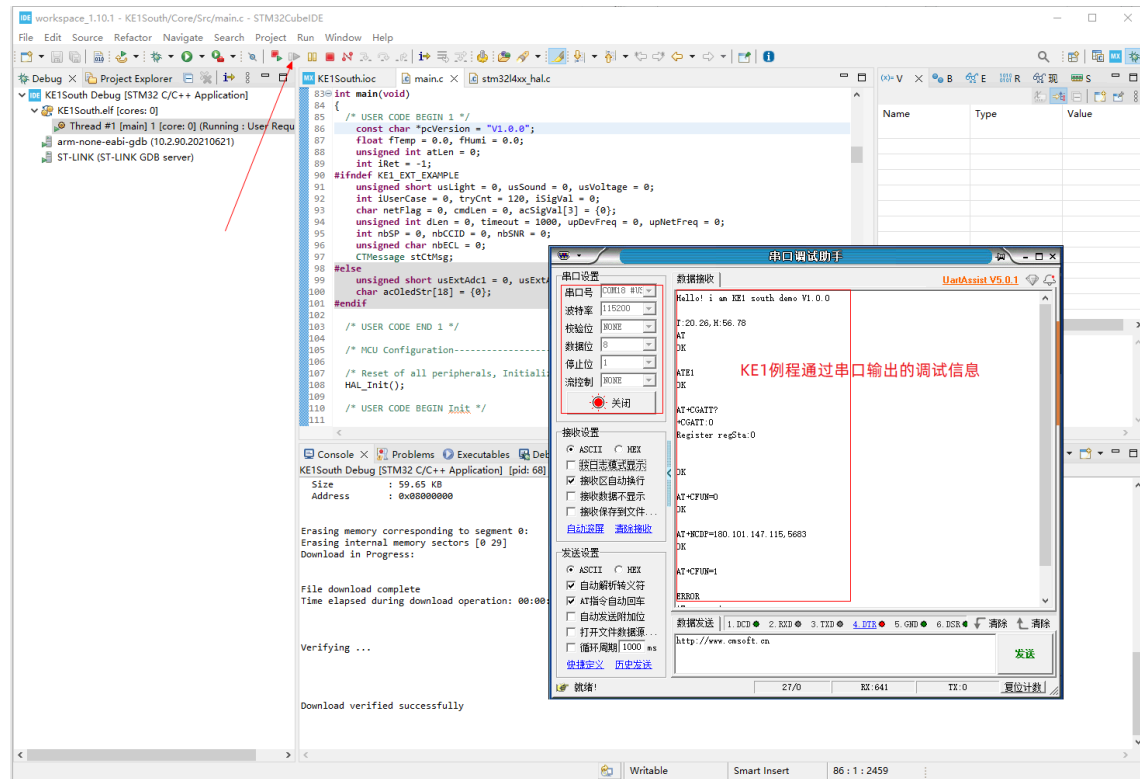
调试界面

成功进入调试后，会自动进入调试界面



运行程序

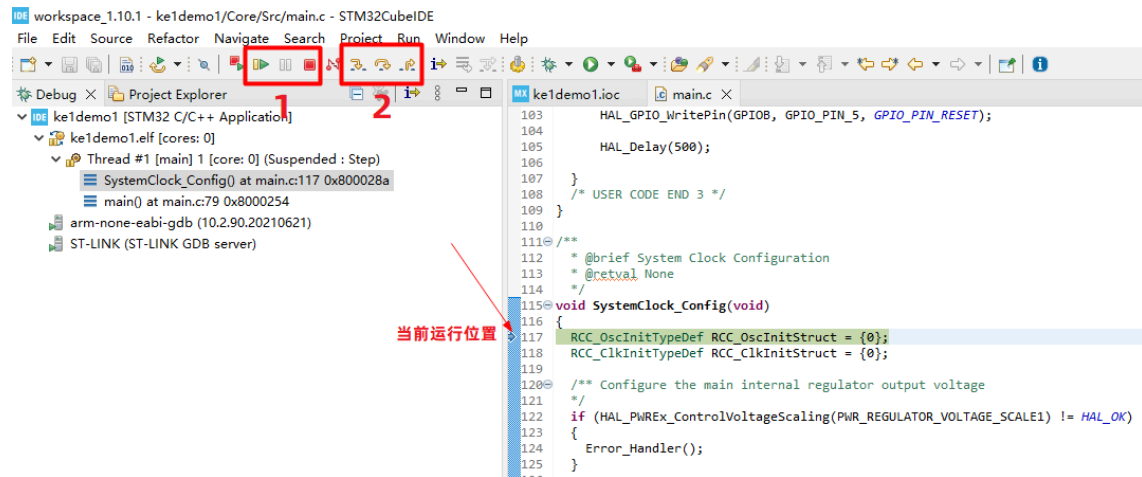
启动串口调试工具，然后点击调试开始按钮，直接运行程序，通过串口调试工具可查看代码通过串口驱动输出的信息



串口工具使用帮助

1. 下载串口工具 [UartAssist串口调试助手](#)
2. 启动串口工具，设置串口号为KE1串口号（例如：COMx #USB-SERIAL CH340）
3. 设置串口工具波特率为 115200，其它参数默认即可
4. 点击"打开"按钮连接串口，即可接收KE1发送的数据

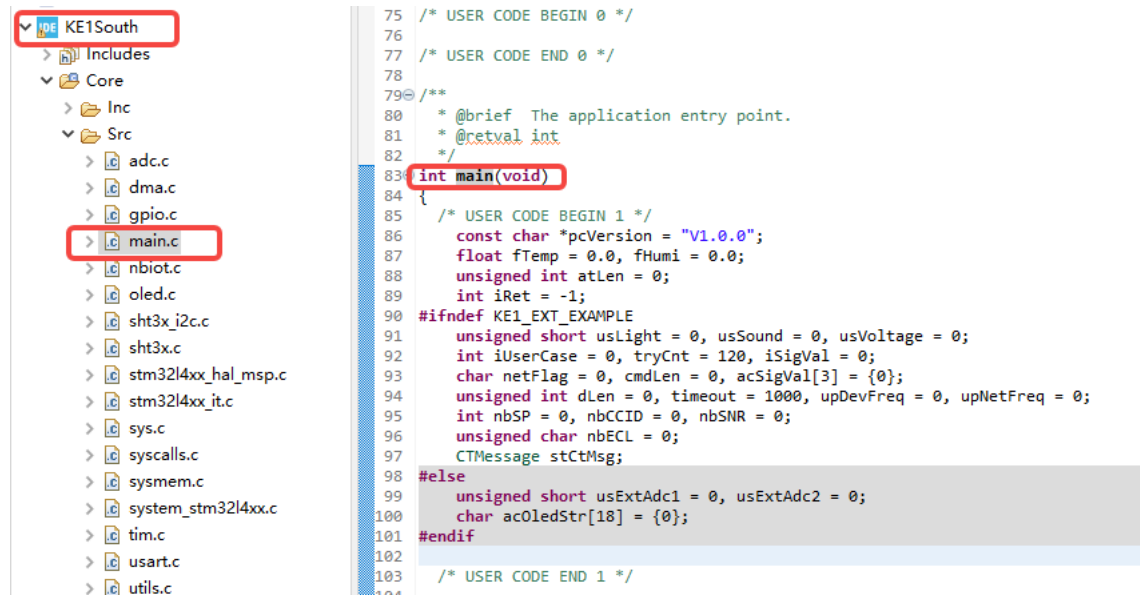
调试命令简介



- 图中标号 1 功能依次为：开始、暂停、结束
- 图中标号 2 功能依次为：进入函数、跳过函数、跳出函数

例程主函数代码简介

打开工程的主函数main.c。参考下面的说明，快速了解程序功能。



```
int main(void)
```

```
{
```

```
    /* USER CODE BEGIN 1 */
```

```
    const char *pcVersion = "V1.0.0";
```

```
    float fTemp = 0.0, fHumi = 0.0;
```

```
    unsigned int atLen = 0;
```

```
    int iRet = -1;
```

```
#ifndef KE1_EXT_EXAMPLE // 外部扩展实验宏定义，默认未定义，则程序运行
```

```
    unsigned short usLight = 0, usSound = 0, usVoltage = 0;
```

```
    int iUserCase = 0, tryCnt = 120, iSigVal = 0;
```

```
    char netFlag = 0, cmdLen = 0, acSigVal[3] = {0};
```

```
    unsigned int dLen = 0, timeout = 1000, upDevFreq = 0, upNetFreq
```

```
    int nbSP = 0, nbCCID = 0, nbSNR = 0;
```

```
    unsigned char nbECL = 0;
```

```
    CTMessage stCtMsg;
```

```
#else // 定义外部扩展实验宏定义，则只测试外部扩展基础实验，具体查看头文件
    unsigned short usExtAdc1 = 0, usExtAdc2 = 0;
    char acOledStr[18] = {0};
#endif

/* USER CODE END 1 */

/* MCU Configuration-----*/

/* Reset of all peripherals, Initializes the Flash interface and
   HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_DMA_Init();
MX_ADC1_Init();
MX_USART1_UART_Init();
MX_USART3_UART_Init();
```

```

MX_TIM2_Init();
MX_TIM1_Init();
MX_TIM15_Init();
/* USER CODE BEGIN 2 */
    Beep_Switch(1); // 使用PWM驱动蜂鸣器开始发声
    HAL_Delay(500); // 延时500ms
    Beep_Switch(0); // 关闭PWM，蜂鸣器停止发声

    UART_Enable_Receive_IT(); // 使能串口接收中断，开始接收串口3 huart3

    OLED_Init(); // 初始化 OLDE屏幕

    //KE1_I2C_SHT31_Init(); // 未使用
    SHT3X_SetI2cAdr(0x44); // 设置温湿度传感器 SHT3X 的 I2C总线地址
    OLED_DrawLogo(); // 显示bmp单色logo图片
    HAL_Delay(1500); // 延时1.5秒

    OLED_ShowKE1(); // 显示 小熊座KE1
    OLED_ShowString(0, 3, (uint8_t *)pcVersion, 6); // 在屏幕指定坐标
    HAL_Delay(1000); // 延时1秒
    printf("Hello! i am KE1 south demo %s\r\n", pcVersion); // 通过调i
    OLED_ShowString(0, 3, (uint8_t *) "Checking...", 8); // 在屏幕指定
    do{
        HAL_Delay(1000); // 延时1秒

        //KE1_I2C_SHT31(&fTemp, &fHumi); // 未使用
        SHT3X_GetTempAndHumi(&fTemp, &fHumi, REPEATAB_HIGH, MODE_CL
        printf("T:%0.2f,H:%0.2f\r\n", fTemp, fHumi); // 通过调试串口1

```

```

        KE1_Send_AT("AT\r\n"); atLen = sizeof(acAtBuf); // 通过串口3
        iRet = KE1_Recv_AT(acAtBuf, &atLen, 1000); // 通过串口3缓存读
        if(0 == fHumi){    OLED_ShowString(0, 3, (uint8_t *)"SHT3X
        else if(0 == iRet){OLED_ShowString(0, 3, (uint8_t *)"NB AT
    }while(0 == fHumi || 0 == iRet); // 一直循环, 直到能正常读取数据

//    if(0 == fHumi || 0 == iRet){
//        printf("I2C or AT error , reboot MCU\n");
//        NVIC_SystemReset(); //无法读取I2C或 AT没有响应则 MCU 重启
//    }

    OLED_Clear(); // OLED 清屏

#ifdef KY_005_IR_ENISSION //测试红外线发送扩展实验 的宏定义
    printf("----- Init & Start KY_005_IR_ENISSION\r\n");
    IRMP_DATA irmp_sent_data; // 红外线发送的数据结构
    irsnd_init(); // 红外线发送初始化
    KE1_Ext_IR_Start(); // 红外线持续发送数据
    printf("----- Init IR Done\r\n");
#elif defined KY_022_IR_RECEIVER //测试红外线接收扩展实验 的宏定义
    printf("----- Init & Start KY_022_IR_RECEIVER\r\n");
    IRMP_DATA irmp_rcv_data; // 红外线接收的数据结构
    HAL_TIM_Base_Start_IT(&htim15); // 启动红外线持续接收定时器
    irmp_init(); //红外线接收初始化
#endif

/* USER CODE END 2 */

```

```

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)// 主函数死循环
{
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */

#ifdef KE1_EXT_EXAMPLE // test extend example 测试扩展实验 的宏定义
/* KE1 EXTEND EXAMPLE BEGIN*/

#if defined KY_022_IR_RECEIVER //测试红外线接收扩展实验 的宏定义
if (irmp_get_data (&irmp_recv_data)) {
printf("Name->[%s],Protocol->%02X, Address->%02X, Command
      irmp_protocol_names[irmp_recv_data.protocol],
      irmp_recv_data.protocol,
      irmp_recv_data.address,
      irmp_recv_data.command,
      irmp_recv_data.flags);
snprintf(acOledStr, sizeof(acOledStr), "IR-Cmd:%04X", irm
OLED_ShowString(0, 2, (uint8_t *)acOledStr, strlen(acOled
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_RESET);
HAL_Delay(1000);
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_6, GPIO_PIN_SET);
}
#else
if(HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0) == GPIO_PIN_RESET){/
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_RESET);
}else{

```

```

        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, GPIO_PIN_SET);
    }
#endif

#if defined KY_005_IR_ENISSION //测试红外线发送扩展实验 的宏定义
    if(1 == key1){
        key1 = 0;
        irmp_sent_data.protocol = IRMP_NEC_PROTOCOL;
        irmp_sent_data.address  = 0x00FC;
        irmp_sent_data.command  = 0x0016;
        irmp_sent_data.flags    = 0;

        irsnd_send_data (&irmp_sent_data, TRUE);
        printf("Sent data\r\n");
    }
#else
    #if 1
        KE1_Ext_PWM_Start(3700, 50);// test beep PWM测试, 驱动蜂鸣器
        HAL_Delay(100);
        KE1_Ext_PWM_Stop();
        HAL_Delay(100);
    #else
        KE1_Ext_PWM_Breathing_LED(5);// PWM测试, LED呼吸灯效果演示
    #endif
#endif

HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_15);// GPIO 输出状态反转

KE1_ADC_Extend_Get(&usExtAdc1, &usExtAdc2);// 扩展例程: KY-0

```

```

snprintf(acOledStr, sizeof(acOledStr), "PA5 AD:%04d", usExt
OLED_ShowString(0, 0, (uint8_t *)acOledStr, strlen(acOledSt
snprintf(acOledStr, sizeof(acOledStr), "PA7 AD:%04d", usExt
OLED_ShowString(0, 2, (uint8_t *)acOledStr, strlen(acOledSt
//printf("usExtAdc1:%d, usExtAdc2:%d\n", usExtAdc1, usExtAd

```

```

/* KE1 EXTEND EXAMPLE END*/

```

```

#else //物联网基础例程

```

```

if(0 == netFlag){// 判断NB模组的网络是否注册成功，并在OLED屏幕
    OLED_Show_Note(NULL, 1, tryCnt);
    if(0 != tryCnt) {
        tryCnt--;
    }else{
        OLED_Show_Note("reg err", 0, 0);
        if(NB_OK == nbiot_reboot(10)){
            iUserCase = NB_STEP_BUFF_CLEAR;tryCnt = 120;
        }else{
            NVIC_SystemReset();
        }
    }
}
}else{// 在OLED屏幕上显示网络信号信息
    if(0 < iSigVal && 32 > iSigVal){
        OLED_Show_Note(NULL, 2, iSigVal);// 显示信号强度值
    }else if(99 == iSigVal){
        OLED_Show_Note("no net", 0, 0);// 显示 no net
    }
}
}

```

```

//KE1_I2C_SHT31(&fTemp, &fHumi); // 未使用
SHT3X_GetTempAndHumi(&fTemp, &fHumi, REPEATABLE_HIGH, MODE_CLOCK)
KE1_ADC_Sensor_Get(&usLight, &usSound, &usVoltage); //通过AD
if(0 < fHumi){
    OLED_ShowT_H(fTemp, fHumi); // OLED显示温湿度信息
}

if(1 == key1){ // 判断开发板上的按键 KEY1 是否按动
    key1 = 0; // 复位key状态, 并打印调试信息
    printf("key1 touch!\r\n");
}
if(1 == key2){ // 判断开发板上的按键 KEY2 是否按动
    key2 = 0; // 复位key状态, 并打印调试信息
    printf("key2 touch!\r\n");
}
timeout = 500;

//printf("case %d - %d - %d - %d\r\n", iUserCase, timeout, u
switch(iUserCase){ // 通过 switch case 一步一步的发送AT命令到N
    case NB_STEP_BUFF_CLEAR: /** 清空串口数据 **/
        KE1_Clear_AT_Buf();
        iUserCase = NB_STEP_CHECK_AT;
        continue;
    case NB_STEP_CHECK_AT:
        KE1_Send_AT("ATE1\r\n"); /** 开启模块AT命令回显功能 **/
        break;
    case NB_STEP_CHECK_REG: /** 检查模块是否已经成功联网 **/
        if(NB_OK == nbiot_check_reg(3)){ /** 已经联网成功, 则
            iUserCase = NB_STEP_UP_REG_INFO;

```



```

        netFlag = 1;
    }else{
        iUserCase = NB_STEP_STOP_MODULE;/** 没有联网成功
    }
    break;
case NB_STEP_STOP_MODULE:
    KE1_Send_AT("AT+CFUN=0\r\n");/** 关闭模块，设置模组参
    timeout = 10000;
    break;
case NB_STEP_SET_COAP:
    KE1_Send_AT("AT+NCDP=180.101.147.115,5683\r\n");/**
    break;
case NB_STEP_START_MODULE:
    KE1_Send_AT("AT+CFUN=1\r\n");/** 启动通信模块 **/
    timeout = 10000;
    break;
case NB_STEP_SET_PDP:
    KE1_Send_AT("AT+CGDCONT=1,\"IP\",\"CTNB\"\r\n");/**
    break;
case NB_STEP_SIM_CHECK:/** 检查SIM卡是否存在 **/
    KE1_Send_AT("AT+CIMI\r\n");
    break;
case NB_STEP_START_REG:
    KE1_Send_AT("AT+CGATT=1\r\n");/** 启动网络附着 **/
    break;
case NB_STEP_SET_AUTO_REG:
    KE1_Send_AT("AT+QREGSWT=1\r\n");/** 设置网络自动注册
    break;
case NB_STEP_WAITING_REG_OK:

```

```

        HAL_Delay(1000);
        KE1_Send_AT("AT+CGATT?\r\n");/** 检查网络是否联网成功
        break;

case NB_STEP_UP_REG_INF0:/** 将联网信息(信号强度, CCID, S
    if(1 == netFlag && 0 == upNetFreq){
        nbiot_get_sigval(acSigVal);iSigVal = atoi(ac

        tryCnt = 0;
        OLED_Show_UP_Flag(1);
        /*
        ** 上报的无线参数值必须在数据范围内才算有效数据, 参
        ** 1. 信号强度, 上报范围应在 -140 ~ -40之间
        ** 2. 覆盖等级, 上报范围应在 0~2之间
        ** 3. 信噪比, 上报范围应在 -20~30之间
        ** 4. 小区ID, 上报范围应在 0~2147483647之间
        ** AT样例: AT+NMGS=11,03FFFFFFA608F651550E01
        ** 平台JSON数据格式: {"SignalPower":-90,"CellID
        * */
        memset(acAtBuf, 0, sizeof(acAtBuf));
        nbiot_get_nuestats(&nbSP, &nbSNR, &nbCCID, &nbE

        printf("Signal:%d, %d, %d, %d\r\n", nbSP, nbCCI

        snprintf(acAtBuf, sizeof(acAtBuf), "AT+NMGS=14,
        KE1_Send_AT(acAtBuf);/** 通过串口发送AT+NMGS命令:
        upNetFreq = (60*60)*2;// 设置主程序发送网络信息的
    }

    break;

```

```

case NB_STEP_UP_DEV_INFO:/** 将获取的传感器信息(温湿度、关
    tryCnt++;
    if(10 == tryCnt || (1 == netFlag && 0 == upDevFreq)
        tryCnt = 0;
        nbiot_get_signl_val(acSigVal);iSigVal = atoi(ac
    }

    if(1 == netFlag && 0 == upDevFreq){
        memset(acDevInfo, 0, sizeof(acDevInfo));
        memset(acAtBuf, 0, sizeof(acAtBuf));

        // 将传感器的数据打包成JSON格式
        dLen = snprintf(acDevInfo, sizeof(acDevInfo), "
        printf("%s\r\n", acDevInfo);// 将JSON数据发送到调

        if(0 < fHumi){
            OLED_Show_UP_Flag(1);
            vdAscii2hex(acDevInfo, acHexBuf);//将打包成
            snprintf(acAtBuf, sizeof(acAtBuf), "AT+NMGS
            //printf("%s\r\n", acAtBuf);
            KE1_Send_AT(acAtBuf);// 通过串口发送AT+NMGS命
        }
        upDevFreq = (60*60);// 设置主程序发送传感器信息的
    }
    break;
}

atLen = sizeof(acAtBuf);
iRet = KE1_Recv_AT(acAtBuf, &atLen, timeout);// 主程序持续接

```

```

//printf("RAT:%d-%d\r\n",iRet, timeout);

if(0 == iRet){//AT命令响应超时
    HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_4);
}else if(1 == iRet){//AT命令接收到OK响应
    if(NB_STEP_WAITING_REG_OK > iUserCase){
        iUserCase++;
        //HAL_Delay(1000);
    }
}else if(2 == iRet){//AT命令接收到ERROR响应
    printf("AT error !\r\n");
    if(NB_STEP_START_MODULE == iUserCase || NB_STEP_SIM_CHECK == iUserCase){
        OLED_ShowNote("NO SIM", 0, 0);
    }
    if(NB_STEP_UP_REG_INFO == iUserCase){
        OLED_ShowNote("NO Reg", 0, 0);
    }
    do{ // 出错报警 (LED灯持续闪烁)
        HAL_Delay(200);
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_6);
    }while(1);

}else if(3 == iRet){//接收到模组网络已成功注册的响应
    printf("Net ready !\r\n");
    netFlag = 1;
    OLED_ShowNote("Net OK", 0, 0);
    HAL_Delay(3000);
    iUserCase = NB_STEP_UP_REG_INFO;
}else if(4 == iRet){//AT命令接收到电信物联网云平台下发数据 (收到数据)
    printf("Data received from the cloud platform!\r\n");
    if(NB_STEP_DATA_RECEIVED == iUserCase){
        iUserCase++;
        HAL_Delay(1000);
    }
}
}

//AT命令接收超时
if(5 == iRet){
    printf("AT command reception timeout!\r\n");
    HAL_Delay(1000);
    HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_4);
}

```

```

printf("%s", acAtBuf);
memset(acUserCmd, 0, sizeof(acUserCmd));
cmdLen = nbiot_parse_NNMI(acAtBuf, acUserCmd); // 解析云

if(strstr(acUserCmd, "AAAA0000")){//云平台定义的响应数据,
    printf("device info upload successfully\r\n");
    OLED_Show_UP_Flag(0);
}else if(strstr(acUserCmd, "CCCC0000")){//云平台定义的响
    printf("Module connectivity upload successfully\r\n
    iUserCase = NB_STEP_UP_DEV_INFO;
    OLED_Show_UP_Flag(0);
}else{//其它则为用户通过小程序或者APP发送给设备的命令数据

    printf("user data[%d]:%s\r\n", cmdLen, acUserCmd);
    memset(&stCtMsg, 0, sizeof(CTMessage));
    nbiot_parse_ct_msg(acUserCmd, &stCtMsg); //解析用户命
    printf("CMD[%02X]:%s\r\n", stCtMsg.userCmd, stCtMsg
    /*
    * 解析用户命令执行对应操作, 可自由发挥去控制灯或者继电器
    * TO-DO
    */

    if(CTL_LED == stCtMsg.userCmd){// Three color LED -
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_5|GPIO_PIN_6
    }else if(CTL_LCD == stCtMsg.userCmd){// OLED --- +N
        OLED_Clear();
        OLED_ShowString(0, 2, stCtMsg.userVal, 16);
    }else if(CTL_SWITCH_1 == stCtMsg.userCmd){// 继电器
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_1);

```

```

        }else if(CTL_SWITCH_2 == stCtMsg.userCmd){// 继电器
            HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_3);
        }
        /* 向平台发送响应数据，告知平台已经成功处理了用户命令。 */
        acUserCmd[1] = '2';
        acUserCmd[6] = '0';acUserCmd[7] = '0';acUserCmd[8]
        printf("CmdResp:%s\r\n", acUserCmd);
        snprintf(acAtBuf, sizeof(acAtBuf), "AT+NMGS=%d,%s\r\n",
        KE1_Send_AT(acAtBuf);
        if(CTL_LCD == stCtMsg.userCmd){
            HAL_Delay(3000);
        }
    }

}

}else{
    if(-2 == iRet){// UART error
        NVIC_SystemReset();
    }
}

if(0 != upNetFreq) {
    upNetFreq--;
    if(0 == upNetFreq){iUserCase = NB_STEP_UP_REG_INFO;}
}

if(0 != upDevFreq) upDevFreq--;

#endif
}

```

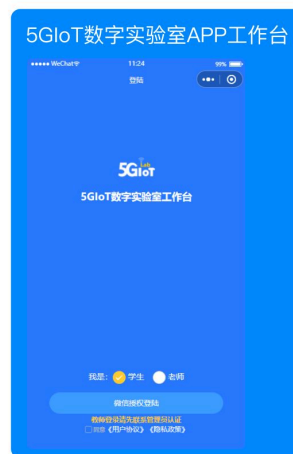
```
/* USER CODE END 3 */
```

```
}
```

体验小程序

扫码登陆小程序，体验更多功能





学生考勤查看

所在班级考勤统计和详情

学生报告查看

学生报告提交统计和详情

学生实训动态浏览

浏览本院系学生提交的实验心得（可点赞，删除，发布）

班级管理

新增，修改班级信息，长按删除

实验室管理

新增，修改实验室信息，长按删除

发布通告

在线发布通告，所在班级学生可在线查看

设备管理

院系所有设备的激活、远程控制、实验数据查看

在线学习

案例指导教学视频

在线签到签退

需前往实验室进行签到、签退

项目查看

查看所选项目详情

成绩查看

查看个人成绩详情

实训动态发布

浏览本院系学生发布的实验心得（可点赞，发布）

实验设备管理

远程控制所绑定的设备和实验数据查看

实验设备绑定

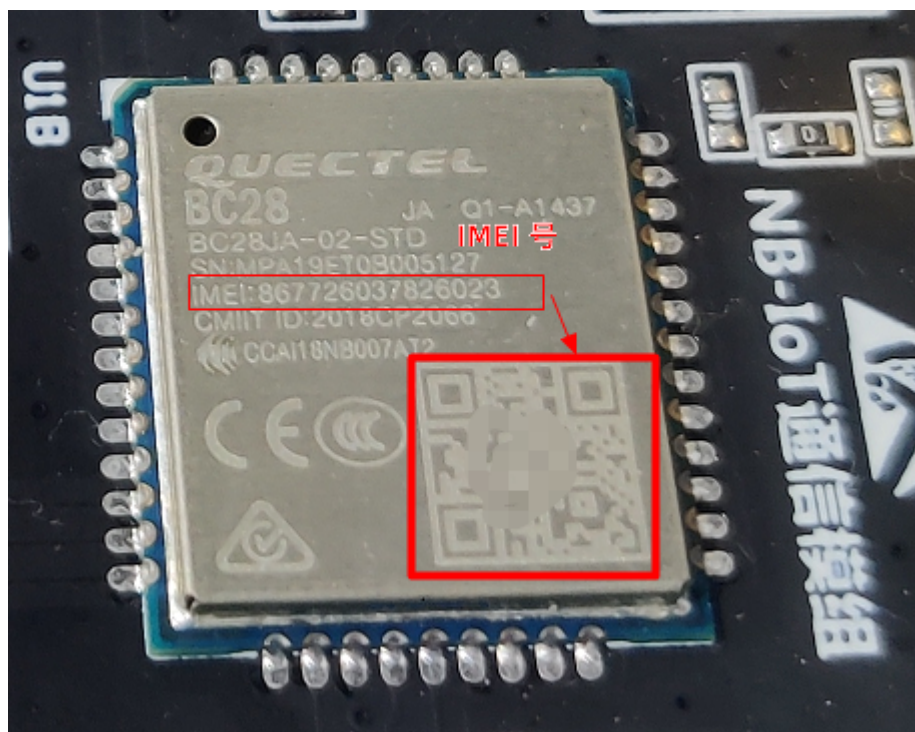
扫描实验设备二维码，绑定设备。方便后续信息统计

在线学习

案例指导教学视频

绑定激活设备

绑定激活设备后，用户可通过小程序远程发送命令数据到 KE1 终端设备。绑定(激活)设备，需要扫描 KE1 通信模组 BC28 的 IMEI号二维码（如下图），但是模组上方的二维码太小，很不容易扫描。



所以建议参考【KE1 AT命令操作】章节获取通信模组IMEI号，或者直接从模组正面上读取，并 [在线生成二维码](#) 进行扫描。



扩展学习分享

1. 嵌入式单片机最主要的开发语言就是C语言，若没有扎实的C语言编程基础，在嵌入式这条路上是走不远的！所以这里推荐几个学习C语言的好网站。
 - C语言在线学习&运行 [菜鸟教程](#)
 - 更深入的了解C语言 [C语言中文网](#)
2. 在物联网云平台开发中，数据的交换常使用的数据格式为JSON。[点击了解JSON](#)，[JSON在线编辑工具](#)

